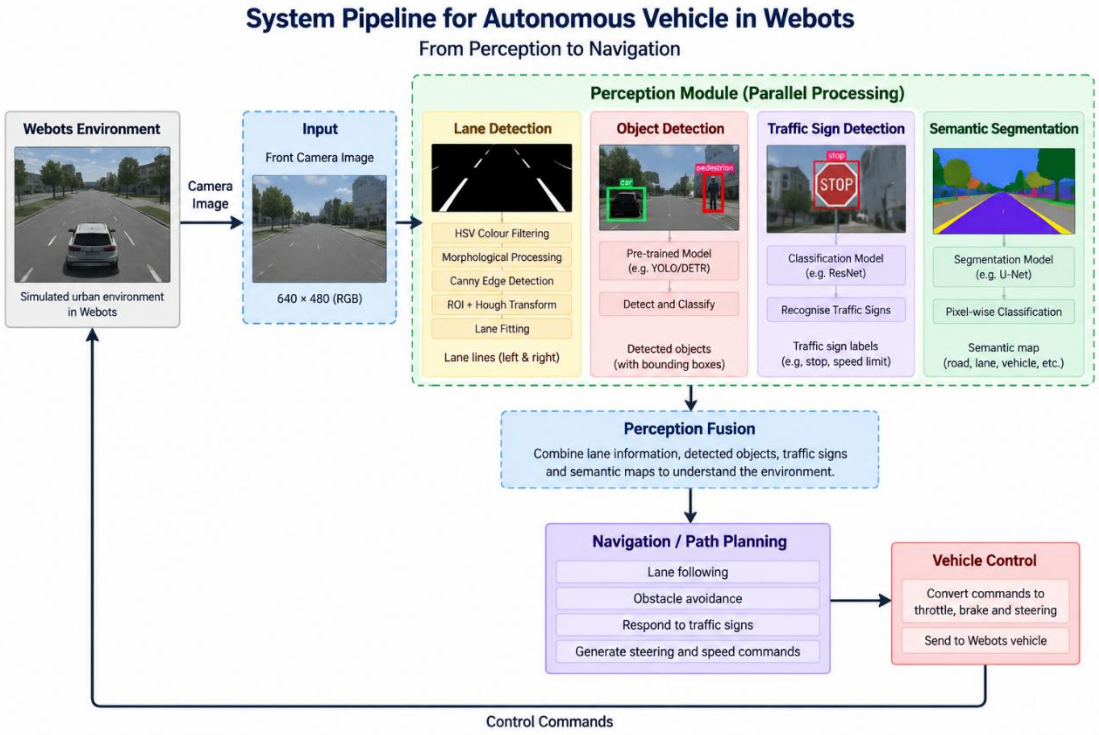
ELEC5308 Assignment 1 – Technical Report

Group 1

Team Module Contributions

Team Member	Main Responsibility
Yue Chang	Lane / Driving-Area Detection
Muhammad Usman Khan	Object Detection / Tracking
Zhiqi Liu	Traffic-Condition Detection
Saifullah Tawsif	Semantic Segmentation
Alaa Aldirani	Navigation / Path Following / Control

1. System Architecture



First, the front camera captures RGB images from Webots, which are processed by four perception modules: lane detection, object detection and tracking, traffic detection, and semantic segmentation. Their outputs are fused to support navigation, which generates steering

and speed commands for vehicle control.

Finally, the generated steering and speed commands are passed to the vehicle control module and applied to the Webots vehicle.

2. Key Design Decisions

2.1 Lane Detection — Yue Chang

For lane detection, we used a traditional computer vision approach instead of a deep learning model to reduce the computational load and keep the system lightweight.

We use HSV filtering, morphological processing, Canny edge detection, ROI and Hough Transform to detect and classify left and right lane lines. When one lane is missing, the system estimates it using the previous centreline or a fixed road width.

2.2 Object Detection — Muhammad Usman Khan

We used a COCO-pretrained Faster R-CNN (ResNet-50 FPN) to detect objects without requiring a labelled training dataset. Detections above 0.5 confidence are returned through the shared ObjectDetectionResult type. Tracking is implemented separately using Hungarian matching based on bounding-box centres.

2.3 Traffic-Condition Detection — Zhiqi Liu

We use HSV colour thresholding to propose candidate regions, followed by ResNet-18 classification across 43 GTSRB classes and pinhole-camera depth estimation. This separates candidate detection from classification and reduces the computational load compared with a heavy end-to-end detector.

2.4 Semantic Segmentation — Saifullah Tawsif

We collected 1,000 synchronized Webots image-mask pairs and trained a U-Net with a ResNet-18 backbone across seven semantic classes. During inference, RGB frames are normalized and processed by the model to generate pixel-level segmentation masks, which are resized to the camera resolution before being passed to downstream modules.

2.5 Navigation / Path Following / Control — Aladdin Dirani

The navigation stage converts detected lane boundaries into path, steering, and speed commands. PathFollower represents each lane boundary as $[x_1, y_1, x_2, y_2]$, identifies near and far points using image y-coordinates, and computes the lane centre by averaging the corresponding left- and right-lane coordinates. The resulting near and far centre points form the path, with the far point used as the navigation target.

Steering is calculated from path geometry using heading error $\theta = \text{atan2}(\Delta, x\Delta y)$, combined with normalized lateral error relative to the image centre. The final controller applies heading and lateral gains of 0.35 and 0.14 respectively, temporal smoothing of 0.7, and clamps steering to ± 0.5 rad to avoid excessive commands.

Speed control uses the same two path points to estimate turn severity using $r = |\Delta x| / \max(|, \Delta y \mid 10^{-6})$. A threshold-based policy commands speeds of 1.0 for $r < 0.15$, 0.7 for $0.15 \leq r < 0.35$, and 0.4 for sharper turns. This couples vehicle speed directly to estimated turn severity while keeping planning, steering, and longitudinal control modular.

3. Evaluation

Lane Detection

We tested the lane detection pipeline using sample road images to evaluate each processing stage. The results show that the system can identify the yellow and white lane markings, apply the ROI, and detect the left and right lane boundaries.

The final output successfully estimates the driving area between the detected lanes. As shown in Figure 1, the intermediate results also allow us to check each stage of the pipeline rather than only the final output.

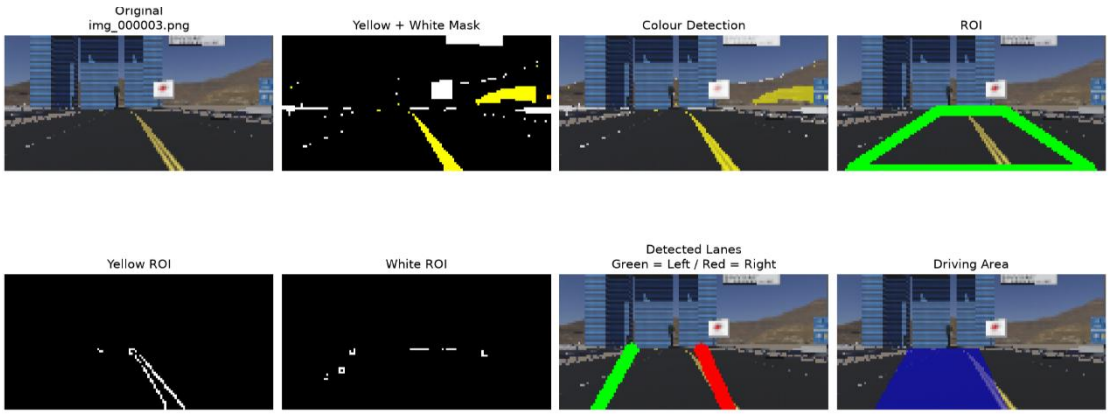


Figure 1. Lane Detection Pipeline and Driving Area Estimation

Object Detection

In city.wbt a pedestrian was detected at 0.99 and held one track ID across hundreds of frames, alongside three tracked stop signs. Ten unit tests pass and the module is ruff-clean. Localisation was accurate throughout, classification was not.

Traffic Detection

The fine-tuned model achieved 100% training accuracy, a peak validation accuracy of 99.97%, and a validation loss of 0.0011 across 15 epochs. Evaluated on 12,630 unseen GTSRB benchmark images, the pipeline demonstrated robust 43-class recognition under variable scales while accurately estimating target distances between 4.7m and 21.8m.

```
(.venv) PS C:\Users\52516\26-S2-Assignment1-Group1> python train.py
Using device: cuda
Starting training loop...
Epoch [1/15] | Train Loss: 0.1990 Acc: 0.9478 | Val Loss: 0.0190 Acc: 0.9955
--> Saved new best model with Validation Accuracy: 0.9955
Epoch [2/15] | Train Loss: 0.0385 Acc: 0.9904 | Val Loss: 0.0868 Acc: 0.9820
Epoch [3/15] | Train Loss: 0.0343 Acc: 0.9911 | Val Loss: 0.0492 Acc: 0.9852
Epoch [4/15] | Train Loss: 0.0119 Acc: 0.9972 | Val Loss: 0.0197 Acc: 0.9945
Epoch [5/15] | Train Loss: 0.0184 Acc: 0.9953 | Val Loss: 0.1032 Acc: 0.9762
Epoch [6/15] | Train Loss: 0.0089 Acc: 0.9979 | Val Loss: 0.0032 Acc: 0.9992
--> Saved new best model with Validation Accuracy: 0.9992
Epoch [7/15] | Train Loss: 0.0079 Acc: 0.9983 | Val Loss: 0.0027 Acc: 0.9992
Epoch [8/15] | Train Loss: 0.0003 Acc: 0.9999 | Val Loss: 0.0018 Acc: 0.9992
Epoch [9/15] | Train Loss: 0.0007 Acc: 0.9999 | Val Loss: 0.0016 Acc: 0.9995
--> Saved new best model with Validation Accuracy: 0.9995
Epoch [10/15] | Train Loss: 0.0001 Acc: 1.0000 | Val Loss: 0.0014 Acc: 0.9995
Epoch [11/15] | Train Loss: 0.0000 Acc: 1.0000 | Val Loss: 0.0014 Acc: 0.9995
Epoch [12/15] | Train Loss: 0.0000 Acc: 1.0000 | Val Loss: 0.0015 Acc: 0.9995
Epoch [13/15] | Train Loss: 0.0000 Acc: 1.0000 | Val Loss: 0.0012 Acc: 0.9995
Epoch [14/15] | Train Loss: 0.0000 Acc: 1.0000 | Val Loss: 0.0011 Acc: 0.9997
--> Saved new best model with Validation Accuracy: 0.9997
Epoch [15/15] | Train Loss: 0.0000 Acc: 1.0000 | Val Loss: 0.0011 Acc: 0.9997
```

```
(.venv) PS C:\Users\52516\26-S2-Assignment1-Group1> python evaluate.py
Initializing TrafficDetector...

Running evaluation on 12630 sample image(s) from data\gtsrb\GTSRB\Final_Test...

Image Name              | Detected Signs (Type / Dist / Conf)          | Conditions
-------------------------------------------------------------------------------------
00000.ppm               | no_heavy_vehicles (11.71m, 0.89)             | Light: normal, Road: clear
00001.ppm               | speed_limit_30 (13.71m, 1.00)                | Light: normal, Road: clear
00002.ppm               | keep_right (13.71m, 1.00)                    | Light: normal, Road: clear
00003.ppm               | turn_right_ahead (16.55m, 1.00)              | Light: normal, Road: clear
00004.ppm               | priority_next_intersection (10.91m, 1.00)    | Light: normal, Road: clear
00005.ppm               | keep_right (12.31m, 1.00)                    | Light: normal, Road: clear
00006.ppm               | general_danger (4.71m, 1.00)                 | Light: normal, Road: clear
00007.ppm               | priority_road (14.55m, 1.00)                 | Light: normal, Road: clear
00008.ppm               | road_work (12.0m, 1.00)                      | Light: normal, Road: clear
00009.ppm               | ahead_only (7.16m, 0.89)                     | Light: normal, Road: clear
00010.ppm               | priority_road (12.97m, 1.00)                 | Light: normal, Road: clear
00011.ppm               | speed_limit_30 (16.55m, 0.69)                | Light: normal, Road: clear
00012.ppm               | slippery_road (6.58m, 1.00)                  | Light: normal, Road: clear
00013.ppm               | speed_limit_100 (12.97m, 1.00)               | Light: normal, Road: clear
00014.ppm               | speed_limit_20 (14.12m, 0.66)                | Light: normal, Road: clear
00015.ppm               | no_passing (21.82m, 1.00)                    | Light: normal, Road: clear
00016.ppm               | double_curve (20.0m, 1.00)                   | Light: normal, Road: clear
00017.ppm               | dangerous_curve_right (14.55m, 1.00)         | Light: normal, Road: clear
00018.ppm               | pedestrians (20.0m, 1.00)                    | Light: normal, Road: clear
00019.ppm               | keep_right (12.0m, 1.00)                     | Light: normal, Road: clear
-------------------------------------------------------------------------------------
```

Figure 2 Traffic detection results

Segmentation

The trained U-Net achieved 95.91% pixel accuracy and 76.42% mean IoU across seven semantic classes on a held-out Webots test set. Excluding the unlearned blue-object class, mean IoU reached 89.16%. Trained for 20 epochs, the model achieved its best validation performance at epoch 14 and successfully generated consistent segmentation masks during live Webots autonomous navigation.

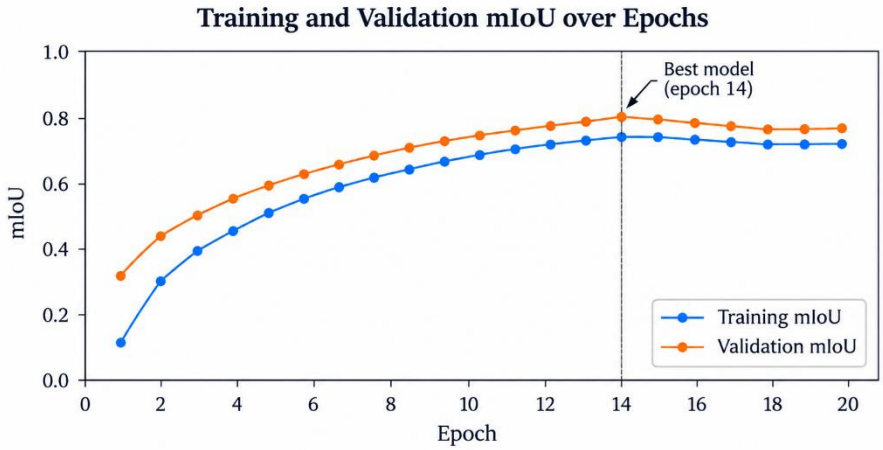


Figure 1. Training and validation mIoU over 20 epochs. The best validation performance was achieved at epoch 14.

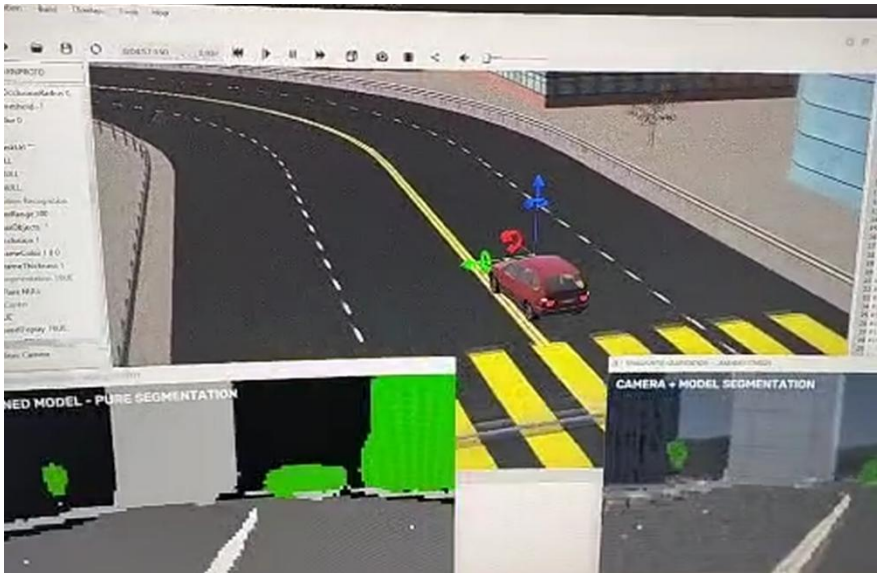


Figure 2: Semantic segmentation demonstration with left recording showing pure segmentation on Webots camera frames and left recording is showing model segmentation overlay over camera

Navigation / Path Following / Control

Webots testing showed that the generated centreline path guided the vehicle through straight and curved roads, with steering correcting path deviations and speed decreasing for sharper turns.

4. Limitations and Future Work

Lane Detection

Limitation:

The main limitation is that fixed image-processing parameters may reduce robustness under different lighting and road conditions.

Future improvement ：

Adaptive thresholds could improve this in future work.

Object Detection

Limitation:

COCO's 80 classes exclude barrels, cones and most road signs, so those are labelled as the nearest known class at high confidence, for example a barrel as a traffic light at 0.91. Raising the threshold does not help.

Future Improvement:

Fine-tune on labelled simulator imagery so the class set matches the environment, and replace centroid matching with a Kalman filter to account for velocity.

Traffic Detection

Limitation:

HSV thresholding struggles under adverse lighting and occlusion, while pinhole estimation assumes fixed sign sizes.

Future improvement ：

Future work could deploy lightweight YOLO models with stereo vision for end-to-end multi-condition robustness.

Segmentation

Limitation:

The main limitation of the segmentation perception was that a limited number of Webots data and mask pairs were collected in one path loop, so the current trained model struggled for different routes and environments.

Future improvement ：

Using large online datasets for training the neural network like Cityscapes should significantly increase adaptability of the perception, as these contain 5000 high resolution images under different lightings and environments.